HTML & CSS

HTML & CSS
PROJECT PORTFOLIO

From First Tags to Advanced Animations

48 Hands-On Projects | 8 Core Modules | 3 Difficulty Tiers

Basic • Intermediate • Advanced — 2 Unique Projects per Level



Complete Front-End Foundation | August 2026

TABLE OF CONTENTS

Module 1: HTML Fundamentals & Document Structure

- [Basic] Personal Bio Page
- [Basic] Recipe Collection Page
- [Intermediate] Multi-Page Travel Blog with Navigation
- [Intermediate] Accessible Data Dashboard (HTML Only)
- [Advanced] HTML Email Template (Table-Based Layout)
- [Advanced] Interactive Documentation Site with HTML5 APIs

Module 2: Text, Typography & CSS Basics

- [Basic] Typography Specimen Sheet
- [Basic] Styled Blog Article Page
- [Intermediate] Magazine-Style Multi-Column Layout
- [Intermediate] CSS-Only Text Effects Gallery
- [Advanced] Fluid Typography System with Clamp()
- [Advanced] Variable Font Playground

Module 3: CSS Selectors & Specificity

- [Basic] Styled Navigation Menu (Selectors Drill)
- [Basic] Zebra-Striped Data Table
- [Intermediate] CSS Specificity Battle Visualizer
- [Intermediate] Form Styling with Advanced Selectors
- [Advanced] Pure CSS Interactive Image Gallery
- [Advanced] CSS-Only Theme Switcher with :has()

Module 4: Box Model & Layout Fundamentals

- [Basic] Box Model Diagram Generator
- [Basic] Pricing Cards with Display Property
- [Intermediate] Sticky Header + Fixed Sidebar + Scrollable Content
- [Intermediate] CSS-Only Dropdown Menu System
- [Advanced] Layered Parallax Scroll Page
- [Advanced] Dashboard Layout with CSS-Only Resizable Panels

Module 5: Flexbox Layout

- [Basic] Responsive Navigation Bar
- [Basic] Holy Grail Layout with Flexbox
- [Intermediate] Photo Gallery with Flex Wrap
- [Intermediate] Testimonial Carousel Cards
- [Advanced] Responsive Dashboard with Nested Flex
- [Advanced] Flexbox-Based Kanban Board

Module 6: CSS Grid Layout

- [Basic] Magazine Homepage Layout
- [Basic] Photo Grid with Spanning Items
- [Intermediate] Responsive Card Grid with Auto-Fit
- [Intermediate] Calendar Month View with Grid
- [Advanced] Full-Page App Layout with Grid Areas

- [Advanced] CSS Grid Art: Geometric Composition

Module 7: Responsive Design & Media Queries

- [Basic] Mobile-First Landing Page
- [Basic] Responsive Image Gallery with srcset
- [Intermediate] Responsive Email Client Layout
- [Intermediate] Dark/Light Mode with prefers-color-scheme
- [Advanced] Container Query Component Library
- [Advanced] Responsive Print Stylesheet for Reports

Module 8: CSS Animations, Transitions & Transforms

- [Basic] Interactive Button Collection
- [Basic] Animated Card Hover Effects
- [Intermediate] CSS Loading Animation Collection
- [Intermediate] Animated Landing Page with Scroll Triggers
- [Advanced] 3D CSS Cube with Interactive Faces
- [Advanced] Animated SVG + CSS Infographic Dashboard

CURRICULUM OVERVIEW

This guide covers the full spectrum of HTML and CSS skills needed to build modern, responsive, accessible websites. The 8 modules progress from raw HTML structure through CSS styling fundamentals, layout systems (Flexbox and Grid), responsive design techniques, and finally advanced animations and 3D transforms. Each module has 6 projects across 3 difficulty tiers.

BASIC	Core syntax and concepts. Single-technique focus, guided output, foundational skills.
INTERMEDIATE	Multiple techniques combined. Real-world scenarios, design decisions, cross-browser awareness.
ADVANCED	Professional-level work. Complex compositions, performance optimization, cutting-edge CSS features.

Module-by-Module Topics:

01	**Module 1: HTML Fundamentals & Document Structure** DOCTYPE, html/head/body, meta tags, semantic elements (header, nav, main, section, article, aside, footer), headings, paragraphs, links, images, comments
02	**Module 2: Text, Typography & CSS Basics** CSS syntax, selectors, properties, font-family, font-size, font-weight, line-height, letter-spacing, text-align, text-decoration, text-transform, color, Google Fonts, @font-face, em/rem/px units
03	**Module 3: CSS Selectors & Specificity** Element, class, ID selectors, descendant/child/sibling combinators, attribute selectors, pseudo-classes (:hover, :focus, :nth-child, :not), pseudo-elements (::before, ::after), specificity calculation, cascade, inheritance
04	**Module 4: Box Model & Layout Fundamentals** Content, padding, border, margin, box-sizing, width/height, min/max dimensions, display (block, inline, inline-block, none), overflow, visibility, float (legacy), positioning (static, relative, absolute, fixed, sticky)
05	**Module 5: Flexbox Layout** display: flex, flex-direction, justify-content, align-items, align-self, flex-wrap, flex-grow/shrink/basis, order, gap, nested flex containers, alignment patterns
06	**Module 6: CSS Grid Layout** display: grid, grid-template-columns/rows, grid-template-areas, fr unit, repeat(), minmax(), auto-fit/auto-fill, grid-column/row span, gap, alignment (justify/align items/content/self), implicit vs explicit grid
07	**Module 7: Responsive Design & Media Queries** Viewport meta tag, media queries (@media), breakpoints, mobile-first design, fluid widths (%, vw, vh, dvh), responsive images (srcset, picture), container queries (@container), clamp(), aspect-ratio, prefers-color-scheme, print styles
08	**Module 8: CSS Animations, Transitions & Transforms** transition (property, duration, timing, delay), @keyframes, animation properties, transform (translate, rotate, scale, skew), transform-origin, perspective, 3D transforms, will-change, prefers-reduced-motion, scroll-driven animations

01 Module 1: HTML Fundamentals & Document Structure

Building well-structured, semantic HTML documents from scratch — the skeleton every web page is built upon.

TOPICS: DOCTYPE, html/head/body, meta tags, semantic elements (header, nav, main, section, article, aside, footer), headings, paragraphs, links, images, comments

BASIC

Basic Projects

Project 1: Personal Bio Page

Create a single-page personal biography using only HTML. Include a proper DOCTYPE declaration, meta charset and viewport tags, headings (h1-h3), paragraphs, an image with alt text, an unordered list of hobbies, and links to social media. No CSS — pure HTML structure.

KEY CONCEPTS PRACTICED:

• DOCTYPE and html/head/body • Meta tags (charset, viewport) • Headings h1-h6 hierarchy • img with alt text, a with href

DELIVERABLE: A single index.html file that renders a complete bio page with correct heading hierarchy, accessible image alt text, and working hyperlinks.

★ **BONUS CHALLENGE:** Add a mailto: link for email contact and a table showing your weekly schedule using <table>, <tr>, <th>, and <td>.

Project 2: Recipe Collection Page

Build an HTML page displaying 3 recipes with proper semantic structure: each recipe inside an <article>, ingredients in <ul>, steps in <ol>, cooking notes in <aside>, and the whole page wrapped in semantic landmarks (<header>, <main>, <footer>). Practice nesting and indentation.

KEY CONCEPTS PRACTICED:

• Semantic elements (article, aside, section) • Ordered vs unordered lists • Nesting elements correctly • header, main, footer landmarks

DELIVERABLE: A well-indented HTML page with 3 recipe articles, each containing a heading, image, ingredient list, numbered steps, and a side note.

★ **BONUS CHALLENGE:** Add a navigation bar (<nav>) at the top with anchor links (#recipe1, #recipe2, #recipe3) that jump to each recipe using id attributes.

INTERMEDIATE

Intermediate Projects

Project 3: Multi-Page Travel Blog with Navigation

Build a 4-page travel blog (Home, Destinations, Gallery, Contact) with consistent navigation across all pages using relative links. Include a working contact form with <form>, <input>, <textarea>, <select>, and <button>. Use <figure> and <figcaption> for gallery images.

KEY CONCEPTS PRACTICED:

• Multi-page linking with relative paths • Form elements (input types, textarea, select) • figure and figcaption for images • Consistent navigation across pages

DELIVERABLE: 4 HTML files linked together with a nav bar, a destinations page with 3 sections, a 6-image gallery, and a contact form with validation attributes.

★ BONUS CHALLENGE: Add HTML5 input validation (required, pattern, minlength) to the contact form and use <details>/<summary> for expandable FAQs on the home page.

Project 4: Accessible Data Dashboard (HTML Only)

Create an HTML-only dashboard displaying data in three tables (Sales by Region, Monthly Revenue, Top Products) with proper <thead>, <tbody>, <tfoot>, scope attributes, and caption elements. Add a description list (<dl>) for KPI definitions. Focus entirely on semantic HTML and accessibility.

KEY CONCEPTS PRACTICED:

• Complex table structure (thead, tbody, tfoot) • Table accessibility (scope, caption) • Description lists (dl, dt, dd) • ARIA landmarks and roles basics

DELIVERABLE: A dashboard page with 3 fully accessible tables, a KPI glossary, and correct use of ARIA roles for screen reader compatibility.

★ BONUS CHALLENGE: Add a <progress> bar for targets, a <meter> element for gauges, and embed an SVG icon inline within a heading.

ADVANCED Advanced Projects

Project 5: HTML Email Template (Table-Based Layout)

Build a responsive HTML email template using only tables for layout (as email clients don't support modern CSS). Include a header with logo, hero section, 3-column feature grid, call-to-action button, and footer — all using nested tables with inline styles. Learn why email HTML is a different world.

KEY CONCEPTS PRACTICED:

• Table-based layout for email clients • Inline styles (email CSS limitations) • Cross-client compatibility considerations • Image hosting and alt text fallbacks

DELIVERABLE: A complete email template that renders correctly in Gmail, Outlook, and Apple Mail, with a 600px max-width table layout and inline styles.

★ BONUS CHALLENGE: Add a dark mode media query inside a <style> tag in <head> (for clients that support it) and provide fallback background colors for Outlook.

Project 6: Interactive Documentation Site with HTML5 APIs

Build a single-page documentation site using advanced HTML5: <details>/<summary> for collapsible sections, <dialog> for modal popups, <template> for reusable content blocks, <output> for live calculations, contenteditable for inline editing, and the <datalist> for searchable input suggestions.

KEY CONCEPTS PRACTICED:

• <details>/<summary> collapsible content • <dialog> element for modals • <template> for reusable HTML fragments • contenteditable and <output> elements

DELIVERABLE: A docs site with 6 collapsible sections, a search input with datalist suggestions, a popup dialog for glossary terms, and an editable code example block.

★ **BONUS CHALLENGE:** Use the <slot> concept manually (via data attributes) to create a simple component-like pattern using only <template> and minimal JavaScript.

02 Module 2: Text, Typography & CSS Basics

Styling text beautifully with CSS typography properties, understanding units, and establishing a visual hierarchy with type alone.

TOPICS: CSS syntax, selectors, properties, font-family, font-size, font-weight, line-height, letter-spacing, text-align, text-decoration, text-transform, color, Google Fonts, @font-face, em/rem/px units

BASIC Basic Projects

Project 1: Typography Specimen Sheet

Create a page that showcases a Google Font pairing (one serif, one sans-serif) at every heading level (h1-h6), paragraph sizes, bold/italic/underline variations, and all text-transform options. Use CSS to set font-family, font-size in rem, line-height, letter-spacing, and color.

KEY CONCEPTS PRACTICED:

• Linking Google Fonts via <link> • font-family, font-size, font-weight • rem vs em vs px units • line-height and letter-spacing

DELIVERABLE: A specimen page showing 2 fonts across all heading levels, 3 paragraph sizes (sm/md/lg), weight variations (300-900), and text transforms.

★ **BONUS CHALLENGE:** Add a CSS custom properties section: define --font-heading, --font-body, --size-base and use them throughout, so changing one variable reskins the entire page.

Project 2: Styled Blog Article Page

Take a plain HTML blog article (800+ words with headings, paragraphs, blockquotes, lists, and a code block) and style it with CSS for maximum readability. Set max-width, comfortable line-length (60-80 characters), proper heading spacing, blockquote styling, and inline code formatting.

KEY CONCEPTS PRACTICED:

• CSS linking (external stylesheet) • Text-align and text-indent • Styling blockquotes and lists • Color and background-color

DELIVERABLE: A beautifully readable article page with clear heading hierarchy, styled blockquotes with left border, inline code backgrounds, and a print-friendly color scheme.

★ **BONUS CHALLENGE:** Add a dark mode toggle using a CSS class (.dark-mode) that swaps all colors, with a button in the HTML that uses a one-line JavaScript toggle.

INTERMEDIATE Intermediate Projects

Project 3: Magazine-Style Multi-Column Layout

Create a magazine article page using CSS multi-column layout (column-count, column-gap, column-rule) with a large drop cap (:first-letter pseudo-element), pull quotes styled with ::before/::after pseudo-elements, and a mix of serif body text with sans-serif captions.

KEY CONCEPTS PRACTICED:

• CSS columns (column-count, column-gap) • :first-letter and :first-line pseudo-elements • ::before and ::after for decorative content • Font stacks and fallback fonts

DELIVERABLE: A 2-3 column magazine article with drop cap, styled pull quotes floating in the text, column rules, and responsive single-column on small screens.

★ **BONUS CHALLENGE:** Use @font-face to load a custom font file locally and apply CSS font-feature-settings to enable ligatures and small caps.

Project 4: CSS-Only Text Effects Gallery

Build a gallery page displaying 10 pure CSS text effects: gradient text (background-clip), text-shadow glow, 3D text with layered shadows, outlined text (-webkit-text-stroke), animated typewriter, text reveal on hover, text masking with images, truncation with ellipsis, vertical text (writing-mode), and highlighted text effect.

KEY CONCEPTS PRACTICED:

• text-shadow for depth and glow • background-clip: text for gradients • -webkit-text-stroke for outlines • text-overflow: ellipsis truncation

DELIVERABLE: A showcase page with 10 labeled text effect cards, each with the CSS code displayed below it, using only CSS (no JavaScript).

★ **BONUS CHALLENGE:** Add a neon sign effect with animated flickering glow using @keyframes and multiple text-shadow layers.

ADVANCED Advanced Projects

Project 5: Fluid Typography System with Clamp()

Build a complete type scale system using CSS clamp() for fluid sizing that smoothly scales between mobile and desktop without breakpoints. Create a modular scale (1.25 ratio), implement fluid spacing to match, and build a reusable utility class system (.text-xs through .text-5xl) that works at any viewport width.

KEY CONCEPTS PRACTICED:

• clamp() for fluid typography • Modular type scales (ratio-based) • CSS custom properties for theming • calc() for dynamic values

DELIVERABLE: A documentation page for your type system showing every size at mobile/tablet/desktop, with CSS variables controlling the entire scale from a single ratio value.

★ **BONUS CHALLENGE:** Add fluid vertical rhythm: make all margins and paddings scale proportionally with the type, maintaining consistent spacing ratios at every viewport width.

Project 6: Variable Font Playground

Create an interactive showcase for a variable font (like Inter, Recursive, or Source Sans) demonstrating all variation axes: weight (wght), width (wdth), slant (slnt), and custom axes. Use CSS font-variation-settings, range inputs for each axis, and show real-time text updates. Display the CSS code needed for each configuration.

KEY CONCEPTS PRACTICED:

• @font-face with variable fonts • font-variation-settings property • CSS custom properties for axes • Optical sizing (font-optical-sizing)

DELIVERABLE: A playground page with sliders controlling font axes, live text preview, CSS output display, and preset buttons for common configurations.

★ **BONUS CHALLENGE:** Animate between font variations on hover using CSS transitions on font-variation-settings (e.g., weight morphing from 100 to 900 on hover).

03 Module 3: CSS Selectors & Specificity

Mastering how CSS targets HTML elements — from basic selectors to complex combinators and the specificity hierarchy that resolves conflicts.

TOPICS: Element, class, ID selectors, descendant/child/sibling combinators, attribute selectors, pseudo-classes (:hover, :focus, :nth-child, :not), pseudo-elements (::before, ::after), specificity calculation, cascade, inheritance

BASIC Basic Projects

Project 1: Styled Navigation Menu (Selectors Drill)

Build a horizontal navigation bar and style it using progressively specific selectors: element selectors for base styles, class selectors for active/inactive states, :hover and :focus pseudo-classes for interaction, descendant selectors for nested dropdowns, and ::after for arrow indicators. Each selector type is visibly labeled.

KEY CONCEPTS PRACTICED:

• Element, class, and ID selectors • :hover and :focus pseudo-classes • Descendant selector (space) vs child (>) • ::after pseudo-element for icons

DELIVERABLE: A nav bar with 5 links, hover effects, an active state class, a dropdown submenu styled via descendant selectors, and arrow indicators via ::after.

★ **BONUS CHALLENGE:** Add :focus-within to highlight the parent menu item when a dropdown link is focused, and use :not(.active) to style all inactive items differently.

Project 2: Zebra-Striped Data Table

Style an HTML table using pseudo-class selectors: :nth-child(even/odd) for alternating row colors, :first-child and :last-child for header/footer styling, :hover for row highlighting, and attribute selectors ([data-status]) for conditional cell coloring (e.g., green for 'active', red for 'inactive').

KEY CONCEPTS PRACTICED:

• :nth-child(even) and :nth-child(odd) • :first-child and :last-child • Attribute selectors ([attr=value]) • Combining multiple selectors

DELIVERABLE: A 10-row data table with zebra stripes, hoverable rows, special first/last row styling, and color-coded status cells based on data attributes.

★ **BONUS CHALLENGE:** Use :nth-child(3n+1) for a 3-color repeating pattern instead of 2, and add :empty to style cells with no data differently.

INTERMEDIATE Intermediate Projects

Project 3: CSS Specificity Battle Visualizer

Create a page with a single HTML element targeted by 10+ conflicting CSS rules (inline style, ID, multiple classes, element, universal, combinators). Display each rule with its specificity score and visually show which rule wins and why. A teaching tool that makes the cascade tangible.

KEY CONCEPTS PRACTICED:

• Specificity calculation (0,0,0,0) • Cascade order and !important • Inheritance vs direct styling • Universal (*) vs element selectors

DELIVERABLE: A visual page showing a target element, all competing rules listed with specificity scores color-coded, and an explanation of the winning rule.

★ **BONUS CHALLENGE:** Add an interactive section where the user adds/removes classes from an element (via checkboxes and minimal JS) and sees the computed style change in real-time.

Project 4: Form Styling with Advanced Selectors

Style a complete registration form using only CSS selectors (no JavaScript): :required and :optional for field importance, :valid and :invalid for real-time validation feedback, :checked for custom checkboxes/radios, :placeholder-shown for floating labels, and + (adjacent sibling) to show/hide help text.

KEY CONCEPTS PRACTICED:

• :valid, :invalid, :required pseudo-classes • :checked for custom checkboxes/radios • :placeholder-shown for floating labels • Adjacent sibling (+) combinator

DELIVERABLE: A styled form with floating labels, green/red borders on valid/invalid fields, custom-styled checkboxes and radio buttons, and help text that appears via CSS only.

★ **BONUS CHALLENGE:** Use :focus-visible to style keyboard focus differently from mouse click focus, and :has() (if supported) to style the form group when its input is valid.

ADVANCED Advanced Projects

Project 5: Pure CSS Interactive Image Gallery

Build an image gallery with filtering, lightbox preview, and category tabs — all using only CSS selectors (zero JavaScript). Use :target for lightbox modals, hidden radio inputs + :checked + general sibling (~) for category filtering, and :nth-of-type for layout variations. A masterclass in selector-driven interactivity.

KEY CONCEPTS PRACTICED:

• :target for URL-driven state • Hidden input + :checked + ~ sibling • General sibling combinator (~) • :nth-of-type for pattern styling

DELIVERABLE: A filterable gallery with 3 categories (Nature, City, Food), clickable thumbnails that open a lightbox overlay, and a grid layout that adapts based on selected category.

★ **BONUS CHALLENGE:** Add a CSS-only slideshow using :checked radio buttons with previous/next navigation, transition effects, and auto-numbered captions via CSS counters.

Project 6: CSS-Only Theme Switcher with :has()

Build a page with 4 complete theme presets (Light, Dark, Ocean, Forest) switchable via radio buttons using :has() selector and CSS custom properties. Each theme changes 15+ properties (colors, fonts, border-radius, shadows, spacing). Demonstrate :has() as the 'parent selector' CSS always lacked.

KEY CONCEPTS PRACTICED:

• :has() selector for parent selection • CSS custom properties for theming • Complex selector chaining • Selector performance considerations

DELIVERABLE: A multi-section landing page that completely reskins (colors, fonts, spacing, shadows, border styles) when a theme radio button is selected, using zero JavaScript.

★ **BONUS CHALLENGE:** Add a :has(:focus-within) rule that highlights the entire form section when any input inside it is focused, and nest :has() with :not() for exclusion-based styling.

04 Module 4: Box Model & Layout Fundamentals

Understanding how every HTML element is a box, how space is calculated around content, and how positioning controls where elements appear.

TOPICS: Content, padding, border, margin, box-sizing, width/height, min/max dimensions, display (block, inline, inline-block, none), overflow, visibility, float (legacy), positioning (static, relative, absolute, fixed, sticky)

BASIC Basic Projects

Project 1: Box Model Diagram Generator

Create a visual representation of the CSS box model using only HTML and CSS: a nested set of divs showing content (blue), padding (green), border (brown), and margin (orange), with pixel values displayed at each layer. Add controls (CSS classes) to toggle between content-box and border-box and see the difference.

KEY CONCEPTS PRACTICED:

• Content, padding, border, margin • box-sizing: content-box vs border-box • Width and height calculations • Background-color for layer visualization

DELIVERABLE: A color-coded box model diagram with labeled pixel values, and two side-by-side versions comparing content-box and border-box sizing.

★ **BONUS CHALLENGE:** Add margin collapsing demonstration: show two stacked boxes where margins collapse, and display the expected vs actual spacing with annotations.

Project 2: Pricing Cards with Display Property

Build a row of 3 pricing cards (Free, Pro, Enterprise) using display: inline-block for horizontal layout. Style each card with padding, borders, border-radius, and box-shadow. Demonstrate the whitespace gap problem with inline-block and three solutions (negative margin, font-size: 0, no HTML whitespace).

KEY CONCEPTS PRACTICED:

• display: block, inline, inline-block • border, border-radius, box-shadow • Padding and margin for spacing • Inline-block whitespace gap issue

DELIVERABLE: 3 side-by-side pricing cards with headers, feature lists, prices, CTA buttons, and a 'recommended' card with a highlighted border. Plus a section explaining the inline-block gap.

★ **BONUS CHALLENGE:** Add a ribbon badge ('Best Value') on the middle card using position: absolute on a rotated element inside a position: relative card.

INTERMEDIATE Intermediate Projects

Project 3: Sticky Header + Fixed Sidebar + Scrollable Content

Build a classic layout with: a sticky top navigation bar (position: sticky), a fixed sidebar menu (position: fixed) that scrolls independently, and a main content area that scrolls beneath them. Handle z-index stacking, overflow-y: auto for the sidebar, and proper body margin to prevent content hiding behind fixed elements.

KEY CONCEPTS PRACTICED:

• position: sticky for scroll-aware headers • position: fixed for persistent elements • z-index and stacking context • overflow-y for scrollable regions

DELIVERABLE: A full-page layout where the header sticks on scroll, the sidebar stays visible and independently scrollable, and main content flows properly without overlap.

★ BONUS CHALLENGE: Add a 'back to top' button using position: fixed at the bottom-right corner, and a position: absolute tooltip that appears on hovering navigation items.

Project 4: CSS-Only Dropdown Menu System

Build a multi-level dropdown navigation (3 levels deep) using position: absolute for dropdowns, position: relative on parent items, :hover to trigger visibility, and proper z-index to layer menus. Handle edge-of-screen overflow and add smooth reveal transitions.

KEY CONCEPTS PRACTICED:

• position: relative/absolute nesting • Top, left, right, bottom offsets • Display: none vs visibility: hidden • Min-width, max-width constraints

DELIVERABLE: A horizontal nav bar with 3 top-level items, each having a dropdown, and at least one dropdown having a sub-dropdown (flyout menu) with smooth CSS transitions.

★ BONUS CHALLENGE: Make the dropdowns work with both :hover (mouse) and :focus-within (keyboard) for full accessibility, and add a mega-menu style dropdown for one item.

ADVANCED Advanced Projects

Project 5: Layered Parallax Scroll Page

Create a parallax scrolling page with 5 content sections and 3 visual layers (background, midground, foreground) using position: fixed and position: relative with z-index layering. Implement CSS perspective and translateZ for true 3D parallax depth, with fallback to simple fixed backgrounds for older browsers.

KEY CONCEPTS PRACTICED:

• perspective and translateZ for parallax • Fixed background attachment • Stacking context hierarchy • Transform-style: preserve-3d

DELIVERABLE: A 5-section scrolling page where background images move at different speeds, creating depth. Includes a hero section, 3 content sections, and a footer, all with layered parallax.

★ BONUS CHALLENGE: Add a CSS-only scroll progress indicator bar at the top using a gradient on a fixed div that grows with page scroll (using body height trick).

Project 6: Dashboard Layout with CSS-Only Resizable Panels

Build a dashboard with multiple panels that can be resized using the CSS resize property and overflow: auto. Include a fixed sidebar, a scrollable main panel, draggable panel dividers (resize: horizontal/vertical), and collapsible sections using max-height transitions. Handle min/max constraints for all panels.

KEY CONCEPTS PRACTICED:

• resize property for user-resizable areas • min-width/max-width/min-height constraints • overflow: auto for scroll containment • Complex nested positioning contexts

DELIVERABLE: A 4-panel dashboard (sidebar, top bar, main content, and right panel) where panels are user-resizable within min/max bounds, with smooth transitions on collapse.

★ **BONUS CHALLENGE:** Use CSS container queries (@container) to make panel content adapt its layout based on the panel's current size rather than the viewport.

05 Module 5: Flexbox Layout

One-dimensional layout mastery with Flexbox — aligning, distributing, and ordering elements along a row or column with precision.

TOPICS: display: flex, flex-direction, justify-content, align-items, align-self, flex-wrap, flex-grow/shrink/basis, order, gap, nested flex containers, alignment patterns

BASIC Basic Projects

Project 1: Responsive Navigation Bar

Build a horizontal navigation bar with a logo on the left, nav links centered, and a 'Sign Up' button pushed to the right using Flexbox. Use justify-content: space-between, align-items: center, and gap for link spacing. The bar should wrap gracefully on narrow screens using flex-wrap.

KEY CONCEPTS PRACTICED:

• display: flex on container • justify-content (space-between, center) • align-items: center for vertical centering • flex-wrap for responsive wrapping

DELIVERABLE: A nav bar with logo, 5 centered links, and a right-aligned button. On narrow screens, links wrap to a second row cleanly.

★ **BONUS CHALLENGE:** Add a search input that flexes to fill available space (flex-grow: 1) between the nav links and the sign-up button.

Project 2: Holy Grail Layout with Flexbox

Build the classic 'Holy Grail' layout: a header (full width), a 3-column middle section (fixed sidebar, flexible main content, fixed aside), and a footer (full width). All using Flexbox. The middle row should stretch to fill available viewport height using flex-grow on the body.

KEY CONCEPTS PRACTICED:

• flex-direction: column for page layout • flex-grow to fill available space • flex-basis for fixed-width sidebars • Nested flex containers

DELIVERABLE: A full-viewport layout with sticky header/footer and a 3-column middle where the center column expands to fill remaining width.

★ **BONUS CHALLENGE:** Make the layout responsive: stack all three columns vertically on mobile using flex-direction: column inside a media query, with order to put the main content first.

INTERMEDIATE Intermediate Projects

Project 3: Photo Gallery with Flex Wrap

Create a responsive photo gallery where images of varying aspect ratios fit into a clean masonry-like grid using Flexbox. Images should grow to fill rows (flex-grow), maintain aspect ratios, and create visually pleasing layouts at any viewport width. Handle the last row's orphan items gracefully.

KEY CONCEPTS PRACTICED:

• flex-wrap: wrap for multi-row layouts • flex-grow and flex-basis for proportional sizing • Object-fit: cover for image cropping • Gap property for consistent spacing

DELIVERABLE: A responsive gallery of 12+ images that form clean rows of varying item counts, with each row perfectly justified and images cropped attractively.

★ BONUS CHALLENGE: Add a flex-based filter bar above the gallery where clicking a category button rearranges visible items using the order property and opacity transitions.

Project 4: Testimonial Carousel Cards

Build a horizontal scrollable testimonial section with cards that snap into place using Flexbox, overflow-x: auto, and scroll-snap-type. Each card has a quote, author photo, name, and star rating. Cards should be equal height using align-items: stretch, with the quote section expanding via flex-grow.

KEY CONCEPTS PRACTICED:

• Horizontal scrolling with flex-wrap: nowrap • align-items: stretch for equal height • flex-shrink: 0 for fixed card widths • Scroll-snap with flex layout

DELIVERABLE: A horizontally scrollable row of 6 testimonial cards that snap into place, all equal height, with flexible quote areas and fixed-height author sections.

★ BONUS CHALLENGE: Add CSS-only previous/next arrows using scroll-behavior: smooth and anchor links that shift the scroll position by one card width.

ADVANCED Advanced Projects

Project 5: Responsive Dashboard with Nested Flex

Build a complete analytics dashboard with 4 layout zones: a collapsible sidebar (flex-basis + transition), a top stats bar (4 metric cards in a row), a main chart area, and a bottom panel with 3 sub-widgets. Every zone uses nested Flexbox. The sidebar collapses to icon-only width, and the entire layout reflows across 3 breakpoints.

KEY CONCEPTS PRACTICED:

• Deeply nested flex containers (3+ levels) • flex-basis transitions for animated collapse • align-self for individual item alignment • Order property for responsive reordering

DELIVERABLE: A 3-breakpoint responsive dashboard where every panel resizes, reorders, or collapses smoothly. Sidebar transitions from full to icon-width, cards reflow from 4-across to 2x2 to stacked.

★ BONUS CHALLENGE: Use flex-grow ratios (2:1, 3:1) to make the chart area proportionally larger than side widgets, and implement a CSS-only accordion in the sidebar using flex and max-height.

Project 6: Flexbox-Based Kanban Board

Create a Trello-style Kanban board with 4 columns (To Do, In Progress, Review, Done). Each column is a flex container (direction: column), cards use flex-shrink: 0, columns stretch to equal height using align-items: stretch on the row, and the board scrolls horizontally on overflow. Include a card count badge and empty-state styling.

KEY CONCEPTS PRACTICED:

• Flex direction: row + column nesting • flex-shrink: 0 for non-collapsing cards • Overflow-x: auto for horizontal scroll • Min-width to prevent column collapse

DELIVERABLE: A 4-column Kanban board with 3-5 cards per column, equal-height columns, horizontal scroll on narrow screens, card count badges, and styled empty columns.

★ **BONUS CHALLENGE:** Add drag-and-drop visual states using CSS only: a :hover + ::after placeholder card in each column showing where a card would drop, and a 'dragging' class style.

06 Module 6: CSS Grid Layout

Two-dimensional layout mastery with CSS Grid — creating complex, responsive page structures with precise row and column control.

TOPICS: display: grid, grid-template-columns/rows, grid-template-areas, fr unit, repeat(), minmax(), auto-fit/auto-fill, grid-column/row span, gap, alignment (justify/align items/content/self), implicit vs explicit grid

BASIC Basic Projects

Project 1: Magazine Homepage Layout

Build a magazine-style homepage using CSS Grid with named grid-template-areas: a header spanning full width, a featured article taking 2/3 width, a sidebar taking 1/3, a 3-column 'latest stories' row, and a footer. Use the fr unit for proportional sizing and gap for gutters.

KEY CONCEPTS PRACTICED:

• grid-template-areas for named layouts • grid-template-columns with fr unit • gap for row and column gutters • grid-area to place items by name

DELIVERABLE: A magazine homepage with a hero header, large featured article beside a sidebar, a 3-card latest stories row, and a footer — all defined via grid-template-areas.

★ **BONUS CHALLENGE:** Rearrange the layout for mobile using a different grid-template-areas definition inside a media query, putting the sidebar below the main content.

Project 2: Photo Grid with Spanning Items

Create a 4-column photo grid where some images span 2 columns, some span 2 rows, and one 'featured' image spans 2x2. Use grid-column and grid-row with span keyword. Set all cells to equal aspect ratio using aspect-ratio property and object-fit: cover on images.

KEY CONCEPTS PRACTICED:

• grid-column: span 2 for multi-column items • grid-row: span 2 for multi-row items • repeat() function for column definition • Aspect-ratio and object-fit in grid cells

DELIVERABLE: A 12-image grid with 3 different span patterns (2-col, 2-row, 2x2), consistent gaps, and properly cropped images filling each cell.

★ **BONUS CHALLENGE:** Make the grid auto-responsive using grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)) so it reflows without media queries.

INTERMEDIATE Intermediate Projects

Project 3: Responsive Card Grid with Auto-Fit

Build a product card grid that automatically adjusts the number of columns based on viewport width using repeat(auto-fit, minmax(280px, 1fr)). Each card has an image, title, description, price, and button, all internally laid out with Grid. Add a 'featured' card that spans 2 columns on wide screens.

KEY CONCEPTS PRACTICED:

• auto-fit vs auto-fill difference • minmax() for responsive columns • Nested grids (grid inside grid items) • Implicit grid rows with grid-auto-rows

DELIVERABLE: A product grid that shows 4 columns on desktop, 3 on tablet, 2 on mobile — all without media queries. Plus a featured card spanning 2 columns with a conditional single-column fallback.

★ **BONUS CHALLENGE:** Use subgrid (grid-template-columns: subgrid) to align the price and button rows across all cards in the same row to the same baseline.

Project 4: Calendar Month View with Grid

Build a functional calendar month view using CSS Grid: 7 columns for days of the week, automatic row creation for weeks, day numbers positioned with grid-column-start for the correct starting day, and today's date highlighted. Events are positioned within day cells using nested flex/grid.

KEY CONCEPTS PRACTICED:

• 7-column grid for calendar layout • grid-column-start for day offset • grid-auto-rows for dynamic weeks • Alignment within grid cells

DELIVERABLE: A monthly calendar for the current month with correct day positioning, day-of-week headers, today highlighted, 3 sample events placed in day cells, and previous/next month grayed-out days.

★ **BONUS CHALLENGE:** Add multi-day events that span across columns using grid-column: span N, styled as colored bars stretching across multiple days.

ADVANCED Advanced Projects

Project 5: Full-Page App Layout with Grid Areas

Build a complete web application layout: top nav bar, collapsible side menu, breadcrumb bar, main content area with a 2-column sub-layout, and a status bar — all using CSS Grid with named areas. The layout should respond to 3 breakpoints with completely different grid-template-areas at each. Include transitions for the sidebar collapse.

KEY CONCEPTS PRACTICED:

• Complex grid-template-areas (6+ areas) • Responsive area remapping • Grid with min-content and max-content • Combining Grid (page) with Flex (components)

DELIVERABLE: A 3-breakpoint app layout that fundamentally reorganizes: desktop (sidebar + header + 2-col content), tablet (collapsed sidebar + stacked content), mobile (no sidebar, stacked everything with hamburger placeholder).

★ **BONUS CHALLENGE:** Use CSS Grid's masonry layout (grid-template-rows: masonry, if supported) for the content section and provide a fallback grid for browsers that don't support it.

Project 6: CSS Grid Art: Geometric Composition

Create a Mondrian or De Stijl inspired geometric art piece purely with CSS Grid: a complex grid with deliberately irregular rows and columns, items spanning varied amounts, bold primary colors (red, blue, yellow) with thick black borders. The composition should change at 3 viewport sizes, becoming a different artwork each time.

KEY CONCEPTS PRACTICED:

• Irregular grid track sizing (mixed units) • Precise item placement with line numbers • Grid lines: named lines for complex placement • Grid as an artistic layout tool

DELIVERABLE: A Mondrian-style artwork using 20+ grid items with varying spans, 3 responsive compositions, thick black grid gaps, and primary color fills.

★ BONUS CHALLENGE: Add a CSS animation that subtly shifts grid items on hover (using grid-column/row transitions where supported) or fades colors with transitions, making the art interactive.

07 Module 7: Responsive Design & Media Queries

Building websites that look and function beautifully on every device — from mobile phones to ultrawide monitors — using responsive techniques.

TOPICS: Viewport meta tag, media queries (@media), breakpoints, mobile-first design, fluid widths (%, vw, vh, dvh), responsive images (srcset, picture), container queries (@container), clamp(), aspect-ratio, prefers-color-scheme, print styles

BASIC Basic Projects

Project 1: Mobile-First Landing Page

Build a product landing page using a mobile-first approach: design for 375px first, then add min-width media queries at 768px and 1024px. The layout transforms from single-column (mobile) to 2-column (tablet) to full desktop layout. Use relative units (%, vw, rem) throughout — zero fixed pixel widths.

KEY CONCEPTS PRACTICED:

• Mobile-first design philosophy • @media (min-width) breakpoints • Viewport units (vw, vh, dvh) • Percentage-based fluid widths

DELIVERABLE: A landing page with hero, features, testimonial, and CTA sections that cleanly adapts across 3 breakpoints using only mobile-first media queries.

★ **BONUS CHALLENGE:** Replace all media queries with CSS clamp() and min()/max() functions to make the layout fully fluid with zero breakpoints.

Project 2: Responsive Image Gallery with srcset

Create a photo gallery that serves different image sizes to different devices using <img srcset> and the <picture> element. Serve small images on mobile (400px wide), medium on tablet (800px), and full-size on desktop (1200px). Add art direction: crop images differently for portrait (mobile) vs landscape (desktop) using <picture> + <source>.

KEY CONCEPTS PRACTICED:

• <img srcset> and sizes attribute • <picture> and <source> elements • Art direction with media in <source> • Loading='lazy' for performance

DELIVERABLE: A gallery with 6 images, each using srcset with 3 sizes and at least 2 images using <picture> for art-directed cropping at different breakpoints.

★ **BONUS CHALLENGE:** Add a CSS aspect-ratio property to reserve space for images before they load, eliminating layout shift (CLS optimization).

INTERMEDIATE Intermediate Projects

Project 3: Responsive Email Client Layout

Build a Gmail-inspired responsive email client: 3-panel desktop (folder sidebar + email list + email preview), 2-panel tablet (folder list + email list), and single-panel mobile (email list only, with slide-in sidebar). Use media queries with Grid/Flex to completely restructure the layout at each breakpoint.

KEY CONCEPTS PRACTICED:

• Complex responsive layout restructuring • Multiple breakpoints (3+ layouts) • Responsive sidebar with transitions • Touch-friendly target sizes (48px min)

DELIVERABLE: A 3-breakpoint email client layout with animated sidebar, appropriately sized touch targets on mobile, hidden panels that could be toggled, and fluid content areas.

★ **BONUS CHALLENGE:** Add prefers-reduced-motion media query to disable all transitions for users who prefer reduced motion, and test with high zoom (200%) to ensure nothing breaks.

Project 4: Dark/Light Mode with prefers-color-scheme

Build a blog page that automatically adapts to the user's OS dark/light mode preference using prefers-color-scheme media query. Define a complete dark and light color system using CSS custom properties. Add a manual toggle (CSS class) that overrides the system preference. Include proper contrast ratios (WCAG AA) for both themes.

KEY CONCEPTS PRACTICED:

• @media (prefers-color-scheme: dark) • CSS custom properties for theming • Color system design (semantic colors) • WCAG contrast ratio compliance

DELIVERABLE: A blog page that auto-detects OS theme, has a manual toggle, uses semantic color variables (--color-text, --color-bg, --color-accent), and passes WCAG AA contrast in both modes.

★ **BONUS CHALLENGE:** Add prefers-contrast: high support that increases borders, removes subtle shadows, and boosts text weight for users who need higher contrast.

ADVANCED Advanced Projects

Project 5: Container Query Component Library

Build 5 components that adapt based on their container size (not viewport) using @container queries: a card (horizontal on wide, vertical on narrow), a nav (full on wide, hamburger on narrow), a sidebar widget, a media object, and a data table. Each component works in any container width.

KEY CONCEPTS PRACTICED:

• @container queries and container-type • Container query units (cqw, cqh) • Component-level responsive design • Container queries vs media queries

DELIVERABLE: 5 self-contained components that resize based on parent container width, demonstrated in a 2-column vs 1-column vs sidebar layout to prove container independence.

★ **BONUS CHALLENGE:** Nest container-query components inside each other (a card inside a sidebar, both responding to their own container) and handle the cascade correctly.

Project 6: Responsive Print Stylesheet for Reports

Take a web-based report page and create a comprehensive print stylesheet using @media print. Handle page breaks (break-before, break-after, break-inside), hide navigation and interactive elements, convert links to show URLs, adjust colors for print (no dark backgrounds), add page margins with @page, and optimize table rendering across page breaks.

KEY CONCEPTS PRACTICED:

• @media print for print styles • @page for page margins and size • break-before/after/inside for pagination • Hiding interactive elements for print

DELIVERABLE: A report page that prints cleanly: navigation hidden, tables don't split mid-row, links show URLs in parentheses, colors are print-optimized, and page headers/footers appear via @page.

★ BONUS CHALLENGE: Add named pages (@page :first for cover, @page :left/:right for alternating margins) and CSS content counters for auto-numbered sections and page references.

08 Module 8: CSS Animations, Transitions & Transforms

Bringing interfaces to life with smooth transitions, complex keyframe animations, and 2D/3D transformations for engaging user experiences.

TOPICS: transition (property, duration, timing, delay), @keyframes, animation properties, transform (translate, rotate, scale, skew), transform-origin, perspective, 3D transforms, will-change, prefers-reduced-motion, scroll-driven animations

BASIC Basic Projects

Project 1: Interactive Button Collection

Create a showcase of 10 animated button styles using CSS transitions: hover scale-up, background color slide, border animation, icon slide-in, underline grow, ripple effect (using ::after), 3D press-down, gradient shift, shadow expansion, and text letter-spacing spread. Each button uses transition for smooth effects.

KEY CONCEPTS PRACTICED:

• transition shorthand (property, duration, timing) • Timing functions (ease, ease-in-out, cubic-bezier) • Transform: scale(), translateX() • ::after for overlay animations

DELIVERABLE: A showcase page with 10 uniquely animated buttons, each labeled with the CSS technique used, all triggered by :hover with smooth transitions.

★ **BONUS CHALLENGE:** Add :focus-visible styles that are distinct from :hover (e.g., a pulsing outline animation) and add transition-delay to create a staggered multi-property animation on one button.

Project 2: Animated Card Hover Effects

Build 6 cards with different hover animations: image zoom with overlay text slide-up, card flip (front/back) using rotateY, tilt effect using perspective + rotateX/Y, image reveal with clip-path, content slide-in from bottom, and border drawing animation. Practice transform-origin and backface-visibility.

KEY CONCEPTS PRACTICED:

• transform: rotate(), scale(), translate() • transform-origin for rotation pivots • backface-visibility for card flips • Overflow: hidden for reveal effects

DELIVERABLE: 6 cards each demonstrating a unique hover animation: zoom, flip, tilt, clip-path reveal, slide, and drawing border. Smooth transitions on all.

★ **BONUS CHALLENGE:** Add a 3D card tilt that follows the mouse position using CSS custom properties set by a tiny JavaScript mouse-tracking snippet (3 lines of JS).

INTERMEDIATE Intermediate Projects

Project 3: CSS Loading Animation Collection

Create 8 loading animations using @keyframes: spinning circle, bouncing dots (3 dots with animation-delay), pulsing ring, progress bar fill, rotating squares, wave bars (5 bars with staggered delays), DNA helix (two rows of dots), and a morphing shape (circle to square to triangle). All pure CSS, no JavaScript.

KEY CONCEPTS PRACTICED:

• @keyframes rule syntax • animation shorthand (name, duration, iteration) • animation-delay for staggered effects • animation-timing-function: steps(), cubic-bezier()

DELIVERABLE: 8 loading spinners/animations on a single page, each labeled with the @keyframes technique used, all running infinitely with smooth looping.

★ **BONUS CHALLENGE:** Create a complex 'typing indicator' animation (3 bouncing dots like iMessage) with realistic timing using custom cubic-bezier curves and precise animation-delay offsets.

Project 4: Animated Landing Page with Scroll Triggers

Build a landing page where elements animate into view as the user scrolls: headings slide in from the left, images fade-and-scale up, stats count up visually (using CSS counters or keyframes), and a timeline draws itself. Use CSS animation-play-state: paused with IntersectionObserver (minimal JS) to trigger animations on scroll.

KEY CONCEPTS PRACTICED:

• @keyframes for complex multi-step animations • animation-fill-mode: forwards/backwards • animation-play-state for scroll triggers • Will-change for GPU acceleration

DELIVERABLE: A 5-section landing page where every section has elements that animate in on scroll: slides, fades, scales, draws, and a stats section with animated number reveals.

★ **BONUS CHALLENGE:** Add a CSS-only approach using scroll-driven animations (@scroll-timeline or animation-timeline: scroll()) for browsers that support it, with the JS trigger as fallback.

ADVANCED Advanced Projects

Project 5: 3D CSS Cube with Interactive Faces

Build a 3D rotating cube using CSS transforms where each face shows different content (image, text, video placeholder, form, chart, about). The cube rotates to the selected face via radio buttons and CSS :checked + transitions. Implement perspective, preserve-3d, rotateX/Y/Z, and translateZ for true 3D positioning.

KEY CONCEPTS PRACTICED:

• perspective and perspective-origin • transform-style: preserve-3d • rotateX(), rotateY(), translateZ() • 3D transforms for spatial positioning

DELIVERABLE: A 3D cube that rotates to show 6 faces when navigation buttons are clicked (CSS-only with radio hack), with smooth rotation transitions and proper 3D perspective.

★ **BONUS CHALLENGE:** Add an auto-rotation animation that spins the cube slowly on both axes, pauses on hover, and snaps to the nearest face when a button is clicked mid-rotation.

Project 6: Animated SVG + CSS Infographic Dashboard

Create an animated dashboard infographic combining CSS animations with inline SVG: animated pie chart (stroke-dasharray + stroke-dashoffset keyframes), bar chart with growing bars (scaleY transform), line chart with drawing effect (stroke-dashoffset animation), a counter with @property-animated numbers, and animated progress rings.

KEY CONCEPTS PRACTICED:

• SVG stroke-dasharray/offset animation • CSS @property for animating custom properties • Transform-origin for chart scaling • Multi-step @keyframes choreography

DELIVERABLE: A dashboard with 5 animated chart widgets that animate in sequence on page load: pie fills, bars grow, line draws, numbers count up, and rings fill. Total animation choreographed over 3 seconds.

★ **BONUS CHALLENGE:** Add scroll-driven animations so each chart section animates when it enters the viewport, and use CSS @property to smoothly animate a number from 0 to a target value with easing.

ALL 48 PROJECTS — QUICK REFERENCE

Complete index of all projects organized by module. Every project is unique and covers distinct HTML/CSS concepts, building from foundational tags through advanced animations.

#	MODULE	LEVEL	PROJECT TITLE
1	HTML Fundamentals & Document	Basic	Personal Bio Page
2	HTML Fundamentals & Document	Basic	Recipe Collection Page
3	HTML Fundamentals & Document	Intermediate	Multi-Page Travel Blog with Navigation
4	HTML Fundamentals & Document	Intermediate	Accessible Data Dashboard (HTML Only)
5	HTML Fundamentals & Document	Advanced	HTML Email Template (Table-Based Layout)
6	HTML Fundamentals & Document	Advanced	Interactive Documentation Site with HTML5 APIs
7	Text, Typography & CSS Basic	Basic	Typography Specimen Sheet
8	Text, Typography & CSS Basic	Basic	Styled Blog Article Page
9	Text, Typography & CSS Basic	Intermediate	Magazine-Style Multi-Column Layout
10	Text, Typography & CSS Basic	Intermediate	CSS-Only Text Effects Gallery
11	Text, Typography & CSS Basic	Advanced	Fluid Typography System with Clamp()
12	Text, Typography & CSS Basic	Advanced	Variable Font Playground
13	CSS Selectors & Specificity	Basic	Styled Navigation Menu (Selectors Drill)
14	CSS Selectors & Specificity	Basic	Zebra-Striped Data Table
15	CSS Selectors & Specificity	Intermediate	CSS Specificity Battle Visualizer
16	CSS Selectors & Specificity	Intermediate	Form Styling with Advanced Selectors
17	CSS Selectors & Specificity	Advanced	Pure CSS Interactive Image Gallery
18	CSS Selectors & Specificity	Advanced	CSS-Only Theme Switcher with :has()
19	Box Model & Layout Fundament	Basic	Box Model Diagram Generator
20	Box Model & Layout Fundament	Basic	Pricing Cards with Display Property
21	Box Model & Layout Fundament	Intermediate	Sticky Header + Fixed Sidebar + Scrollable Content
22	Box Model & Layout Fundament	Intermediate	CSS-Only Dropdown Menu System
23	Box Model & Layout Fundament	Advanced	Layered Parallax Scroll Page
24	Box Model & Layout Fundament	Advanced	Dashboard Layout with CSS-Only Resizable Panels
25	Flexbox Layout	Basic	Responsive Navigation Bar
26	Flexbox Layout	Basic	Holy Grail Layout with Flexbox
27	Flexbox Layout	Intermediate	Photo Gallery with Flex Wrap
28	Flexbox Layout	Intermediate	Testimonial Carousel Cards
29	Flexbox Layout	Advanced	Responsive Dashboard with Nested Flex
30	Flexbox Layout	Advanced	Flexbox-Based Kanban Board
31	CSS Grid Layout	Basic	Magazine Homepage Layout
32	CSS Grid Layout	Basic	Photo Grid with Spanning Items

#	MODULE	LEVEL	PROJECT TITLE
33	CSS Grid Layout	Intermediate	Responsive Card Grid with Auto-Fit
34	CSS Grid Layout	Intermediate	Calendar Month View with Grid
35	CSS Grid Layout	Advanced	Full-Page App Layout with Grid Areas
36	CSS Grid Layout	Advanced	CSS Grid Art: Geometric Composition
37	Responsive Design & Media Qu	Basic	Mobile-First Landing Page
38	Responsive Design & Media Qu	Basic	Responsive Image Gallery with srcset
39	Responsive Design & Media Qu	Intermediate	Responsive Email Client Layout
40	Responsive Design & Media Qu	Intermediate	Dark/Light Mode with prefers-color-scheme
41	Responsive Design & Media Qu	Advanced	Container Query Component Library
42	Responsive Design & Media Qu	Advanced	Responsive Print Stylesheet for Reports
43	CSS Animations, Transitions	Basic	Interactive Button Collection
44	CSS Animations, Transitions	Basic	Animated Card Hover Effects
45	CSS Animations, Transitions	Intermediate	CSS Loading Animation Collection
46	CSS Animations, Transitions	Intermediate	Animated Landing Page with Scroll Triggers
47	CSS Animations, Transitions	Advanced	3D CSS Cube with Interactive Faces
48	CSS Animations, Transitions	Advanced	Animated SVG + CSS Infographic Dashboard

RECOMMENDED TOOLS & RESOURCES

Code Editors
VS Code (with Live Server, Prettier, Emmet), Sublime Text, WebStorm, or CodePen/CodeSandbox for quick prototyping.

Browser DevTools
Chrome DevTools (Elements, Styles, Box Model, Layout panels), Firefox DevTools (excellent CSS Grid/Flex inspectors), Safari Web Inspector for mobile debugging.

CSS Generators
CSS Gradient Generator (cssgradient.io), Box Shadow Generator, Clip-path Maker (bennettfeely.com/clippy), Cubic Bezier Visualizer (cubic-bezier.com).

Validation & Accessibility
W3C HTML Validator (validator.w3.org), W3C CSS Validator (jigsaw.w3.org), axe DevTools for accessibility auditing, Lighthouse for performance.

Design References
MDN Web Docs (comprehensive reference), CSS-Tricks (tutorials and guides), Can I Use (caniuse.com) for browser support checks.

Practice Platforms
Frontend Mentor (real-world challenges), CSS Battle (code golf for CSS), 100 Days of CSS, freeCodeCamp Responsive Web Design certification.